

单周期 RISC-V CPU 波形图验证

仿真环境与配置

项目	配置
仿真工具	Xilinx Vivado 2019.2
仿真类型	功能仿真 (Behavioral Simulation)
时钟周期	10ns (100MHz)
数值显示	十进制 (Decimal)

重要说明： 本验证采用**功能仿真 (Behavioral Simulation)**，直接验证 RTL 代码的逻辑正确性，不涉及时序约束和门级延迟。功能仿真是验证 CPU 设计正确性的标准方法。

测试程序说明

本测试程序包含 10 条指令，覆盖 R 型、I 型、S 型、B 型四种指令格式，验证 CPU 的完整功能。

说明： 波形图中所有数值均以**十进制 (Decimal)** 格式显示，便于直观理解数据含义。机器码和地址在说明中同时提供十六进制表示以对照指令编码规范。

时序特性说明

在单周期 CPU 设计中，**寄存器写回**和**数据存储器写入**均采用**同步时序逻辑**，在时钟上升沿触发更新。这意味着：

1. 寄存器写回时序：

- 当前指令的 ALU 计算结果在本周期内通过组合逻辑传播到寄存器堆的写数据端口
- 实际写入操作在**下一个时钟上升沿**完成
- 因此，寄存器的新值在**下一条指令的波形图**中才能观察到

2. 数据存储器写入时序：

- sw 指令在本周期内计算存储地址并准备写数据
- 实际写入 DM 的操作在**时钟上升沿**同步完成
- 写入的数据在**后续波形图**的 memory 数组中可见

3. 读操作时序：

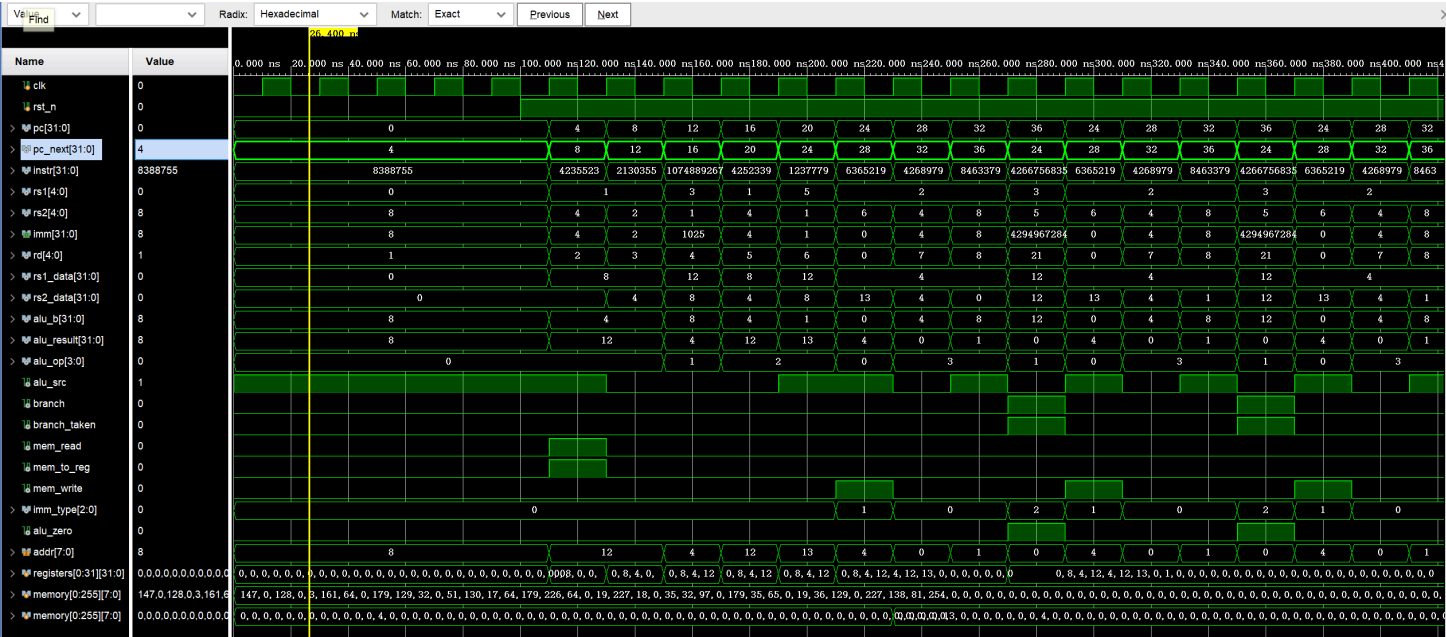
- 寄存器读取和数据存储器读取（lw）均为**组合逻辑**
- 读操作在同一周期内完成，结果立即可用

验证示例：

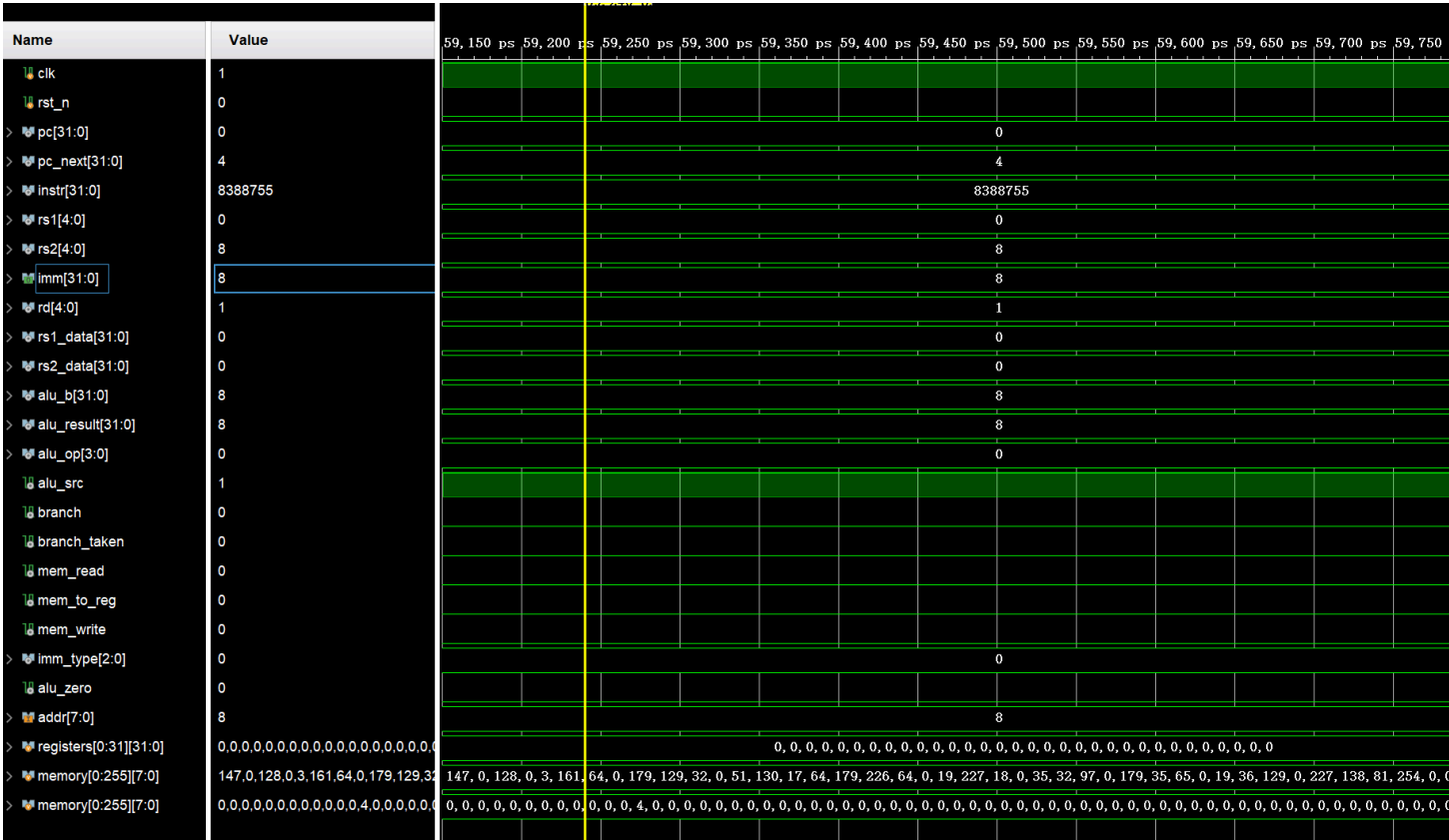
- addi x1, x0, 8（PC=0）执行时，x1 的新值 8 在 lw 指令（PC=4）的波形图中才显示
- sw x6, 0(x2)（PC=24）执行时，DM[4]=13 的写入在下一周期生效
- 这种设计符合标准时序逻辑规范，确保数据稳定性和时序正确性

波形总览图

总览图展示完整的 10 条指令执行过程（数值以十进制显示）：



1. addi x1, x0, 8 (I 型立即数加法)



指令地址: PC = 0x00

机器码: 0x00800093 (十进制: 8388755)

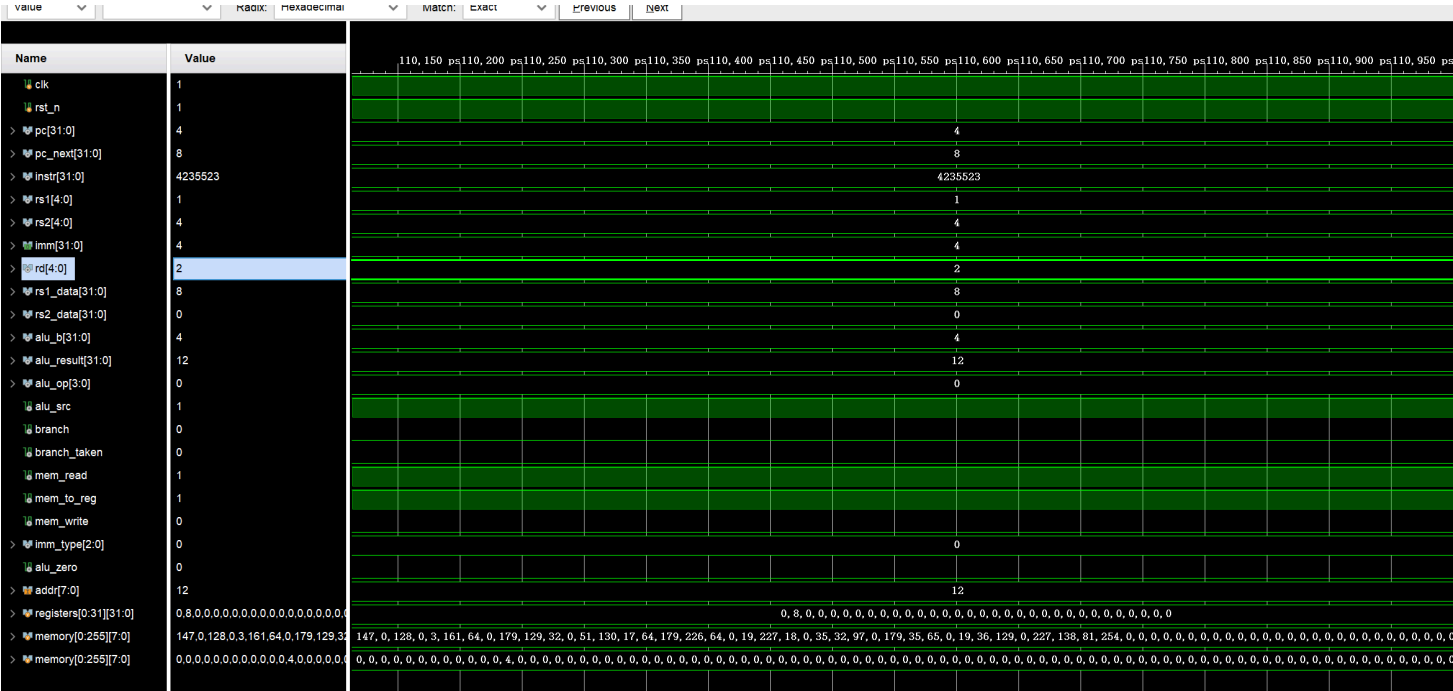
指令功能： $x1 = x0 + 8 = 0 + 8 = 8$

关键信号验证:

- `pc[31:0] = 0`: 当前指令地址
- `pc_next[31:0] = 4`: 下一指令地址 (PC+4)
- `instr[31:0] = 8388755`: 指令机器码
- `rd[4:0] = 1`: 目标寄存器 x1
- `rs1[4:0] = 0`: 源寄存器 x0
- `imm[31:0] = 8`: 立即数值
- `alu_result[31:0] = 8`: ALU 计算结果
- `alu_src = 1`: ALU 第二操作数来自立即数
- `alu_op[3:0] = 0`: ADD 操作
- `reg_write = 1`: 写回寄存器使能
- `mem_read = 0, mem_write = 0`: 不访问内存

测试结果： ✓ 通过 x1 成功写入 8

2. lw x2, 4(x1) (Load 指令)



指令地址：PC = 0x04
机器码：0x0040a103 (十进制: 4235523)
指令功能：x2 = DM[x1 + 4] = DM[8 + 4] = DM[12] = 4

关键信号验证：

- pc[31:0] = 4：当前指令地址
- pc_next[31:0] = 8：下一指令地址
- instr[31:0] = 4235523：lw 指令机器码
- rd[4:0] = 2：目标寄存器 x2
- rs1[4:0] = 1：基址寄存器 x1
- rs1_data[31:0] = 8：x1 的值
- imm[31:0] = 4：偏移量
- alu_result[31:0] = 12：地址计算结果 (8+4)
- alu_src = 1：使用立即数
- mem_read = 1：内存读使能
- mem_to_reg = 1：写回数据来自内存
- reg_write = 1：写回寄存器使能

测试结果：✓ 通过 x2 成功从 DM[12]读取值 4

3. add x3, x1, x2 (R 型加法)



指令地址: PC = 0x08

机器码: 0x002081b3 (十进制: 2130355)

指令功能： $x3 = x1 + x2 = 8 + 4 = 12$

关键信号验证:

- pc[31:0] = 8: 当前指令地址
- instr[31:0] = 2130355: add 指令机器码
- rd[4:0] = 3: 目标寄存器 x3
- rs1[4:0] = 1, rs2[4:0] = 2: 源寄存器
- rs1_data[31:0] = 8, rs2_data[31:0] = 4: 操作数
- alu_b[31:0] = 4: ALU 第二操作数 (来自 rs2)
- alu_result[31:0] = 12: 加法结果
- alu_src = 0: ALU 第二操作数来自寄存器
- alu_op[3:0] = 0: ADD 操作
- reg_write = 1: 写回寄存器

测试结果： ✓ 通过 x3 成功写入 12

4. sub x4, x3, x1 (R 型减法)



指令地址： PC = 0x0C (十进制: 12)
机器码： 0x40118233 (十进制: 1074889267)
指令功能： $x4 = x3 - x1 = 12 - 8 = 4$

关键信号验证：

- pc[31:0] = 12：当前指令地址
- pc_next[31:0] = 16：下一指令地址
- instr[31:0] = 1074889267：sub 指令机器码
- rd[4:0] = 4：目标寄存器 x4
- rs1[4:0] = 3， rs2[4:0] = 1：源寄存器
- rs1_data[31:0] = 12， rs2_data[31:0] = 8：操作数
- alu_result[31:0] = 4：减法结果
- alu_op[3:0] = 1：SUB 操作
- alu_src = 0：使用寄存器操作数
- reg_write = 1：写回寄存器

测试结果： ✓ 通过 x4 成功写入 4

5. or x5, x1, x4 (R 型按位或)



指令地址：PC = 0x10 (十进制: 16)
机器码：0x0040e2b3 (十进制: 4252339)
指令功能：x5 = x1 | x4 = 8 | 4 = 12 (0b1000 | 0b0100 = 0b1100)

关键信号验证：

- pc[31:0] = 16：当前指令地址
- pc_next[31:0] = 20：下一指令地址
- instr[31:0] = 4252339：or 指令机器码
- rd[4:0] = 5：目标寄存器 x5
- rs1[4:0] = 1，rs2[4:0] = 4：源寄存器
- rs1_data[31:0] = 8，rs2_data[31:0] = 4：操作数
- alu_result[31:0] = 12：按位或结果
- alu_op[3:0] = 2：OR 操作
- reg_write = 1：写回寄存器

测试结果：✓ 通过 x5 成功写入 12

6. ori x6, x5, 1 (I 型立即数按位或)

[illegible]

指令地址: PC = 0x14 (十进制: 20)

机器码: 0x0012e313 (十进制: 1237779)

指令功能： $x6 = x5 \mid 1 = 12 \mid 1 = 13$ (0b1100 | 0b0001 = 0b1101)

关键信号验证:

- `pc[31:0] = 20`: 当前指令地址
- `pc_next[31:0] = 24`: 下一指令地址
- `instr[31:0] = 1237779`: ori 指令机器码
- `rd[4:0] = 6`: 目标寄存器 x6
- `rs1[4:0] = 5`: 源寄存器 x5
- `rs2[4:0] = 1`: 立即数字段 (虽然 I 型不使用 rs2, 此处显示 imm 部分)
- `imm[31:0] = 1`: 立即数值
- `rs1_data[31:0] = 12`: x5 的值
- `alu_result[31:0] = 13`: 按位或结果
- `alu_op[3:0] = 2`: OR 操作
- `alu_src = 1`: 使用立即数
- `reg_write = 1`: 写回寄存器

测试结果： ✓ 通过 x6 成功写入 13

7. sw x6, 0(x2) (Store 指令)

[illegible]

指令地址: PC = 0x18 (十进制: 24)

机器码: 0x00612023 (十进制: 6365219)

指令功能： $DM[x2 + 0] = x6$ ，即 $DM[4] = 13$

关键信号验证:

- `pc[31:0] = 24`: 当前指令地址
- `pc_next[31:0] = 28`: 下一指令地址
- `instr[31:0] = 6365219`: sw 指令机器码
- `rs1[4:0] = 2`: 基址寄存器 x2
- `rs2[4:0] = 6`: 源数据寄存器 x6
- `imm[31:0] = 0`: 偏移量
- `rs1_data[31:0] = 4`: x2 的值 (地址)
- `rs2_data[31:0] = 13`: x6 的值 (要写入的数据)
- `alu_result[31:0] = 4`: 地址计算结果 (4+0)
- `alu_src = 1`: 使用立即数计算地址
- `mem_write = 1`: 内存写使能
- `reg_write = 0`: 不写回寄存器

- `imm_type[2:0] = 1`: S 型立即数

测试结果： ✓ 通过 DM[4]成功写入 13（可从 memory 数组看到地址 4 处值为 13）

8. slt x7, x2, x4 (R 型有符号比较)

[illegible]

指令地址: PC = 0x1C (十进制: 28)

机器码: 0x004123b3 (十进制: 4268979)

指令功能： $x7 = (x2 < x4) ? 1 : 0 = (4 < 4) ? 1 : 0 = 0$

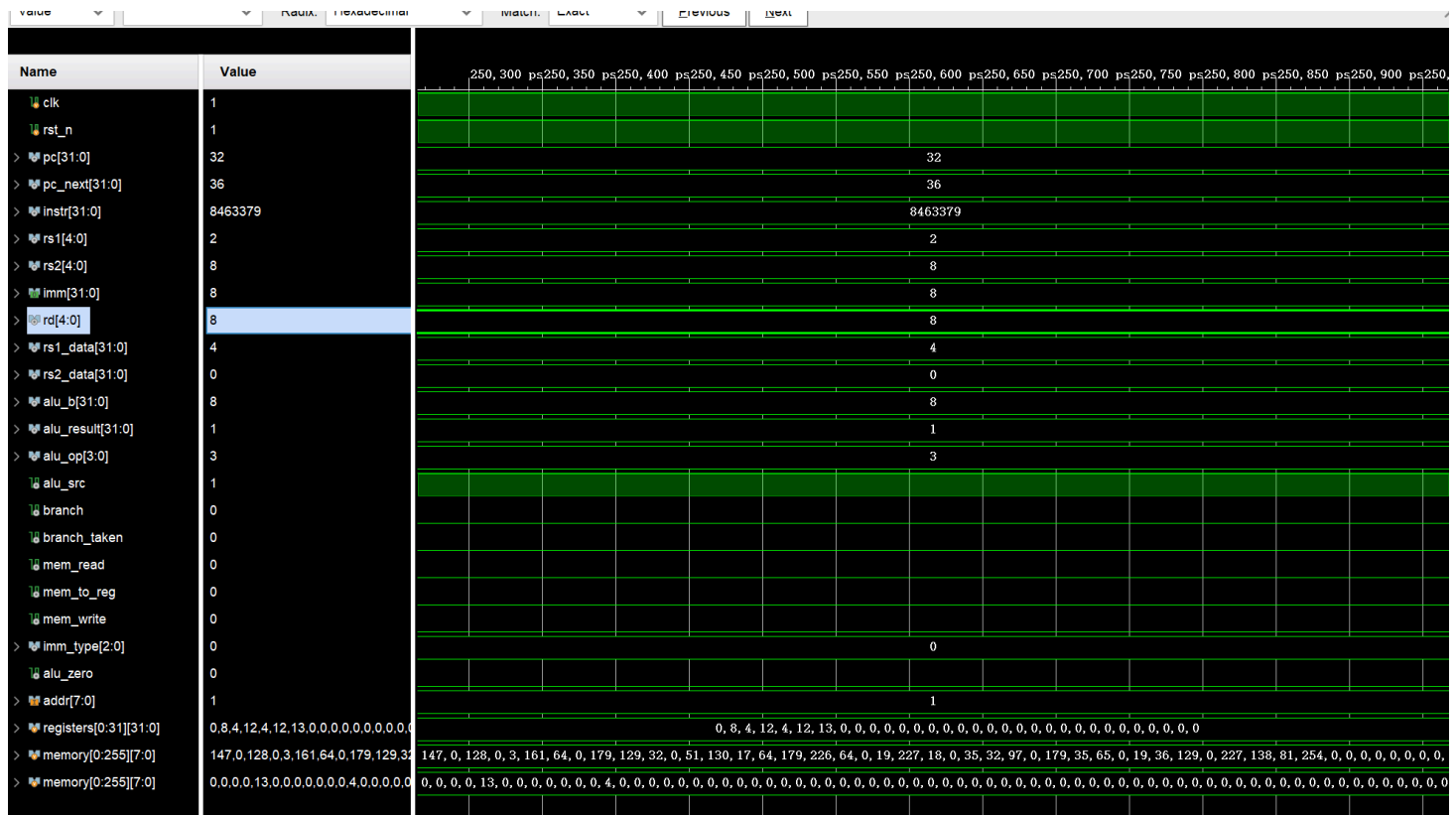
关键信号验证:

- `pc[31:0] = 28`: 当前指令地址
- `pc_next[31:0] = 32`: 下一指令地址
- `instr[31:0] = 4268979`: slt 指令机器码
- `rd[4:0] = 7`: 目标寄存器 x7
- `rs1[4:0] = 2, rs2[4:0] = 4`: 源寄存器
- `rs1_data[31:0] = 4, rs2_data[31:0] = 4`: 比较操作数
- `alu_result[31:0] = 0`: 比较结果 (4 不小于 4)
- `alu_op[3:0] = 3`: SLT 操作
- `alu_src = 0`: 使用寄存器操作数

- `reg_write = 1`: 写回寄存器

测试结果： ✓ 通过 x7 成功写入 0 (4 不小于 4)

9. `slti x8, x2, 8` (I 型有符号立即数比较)



指令地址: PC = 0x20 (十进制: 32)

机器码: 0x00812413 (十进制: 8463379)

指令功能: $x8 = (x2 < 8) ? 1 : 0 = (4 < 8) ? 1 : 0 = 1$

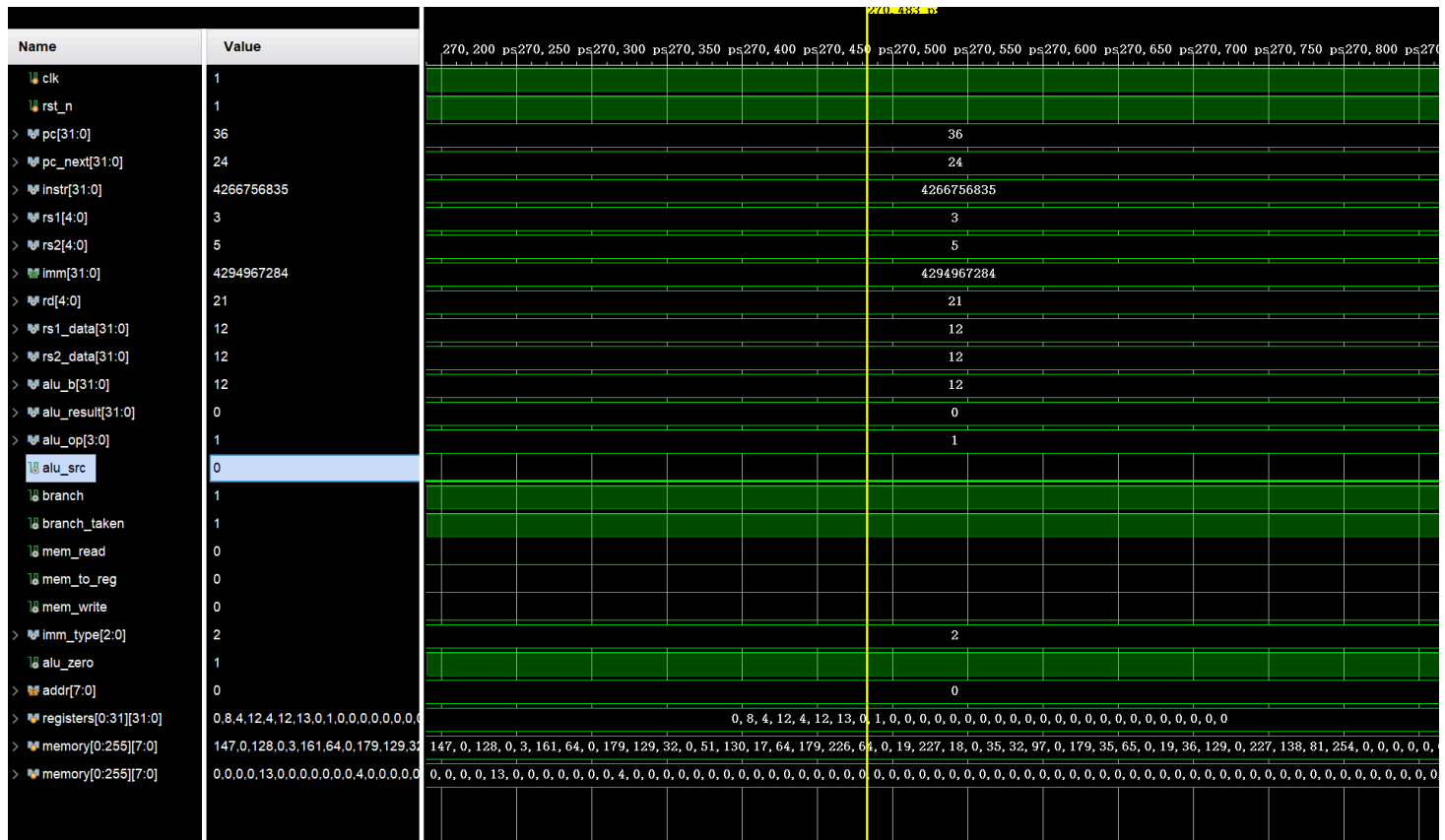
关键信号验证:

- `pc[31:0] = 32`: 当前指令地址
- `pc_next[31:0] = 36`: 下一指令地址
- `instr[31:0] = 8463379`: slti 指令机器码
- `rd[4:0] = 8`: 目标寄存器 x8
- `rs1[4:0] = 2`: 源寄存器 x2
- `imm[31:0] = 8`: 立即数比较值
- `rs1_data[31:0] = 4`: x2 的值
- `rs2_data[31:0] = 0`: I 型指令不使用 rs2
- `alu_result[31:0] = 1`: 比较结果 (4 小于 8)

- `alu_op[3:0] = 3`: SLT 操作
- `alu_src = 1`: 使用立即数
- `reg_write = 1`: 写回寄存器

测试结果： ✓ 通过 x8 成功写入 1 (4 小于 8)

10. beq x3, x5, -12 (B 型条件分支跳转 - 附加分)



指令地址: PC = 0x24 (十进制: 36)

机器码: 0xfe518ae3 (十进制: 4266756835)

指令功能: if (x3 == x5) then PC = PC + (-12) = 36 + (-12) = 24

关键信号验证:

- `pc[31:0] = 36`: 当前指令地址
- `pc_next[31:0] = 24`: 跳转目标地址 ($36-12=24$, 回到 `sw` 指令)
- `instr[31:0] = 4266756835`: `beq` 指令机器码
- `rs1[4:0] = 3`, `rs2[4:0] = 5`: 比较的两个寄存器
- `rs1_data[31:0] = 12`, `rs2_data[31:0] = 12`: 比较值 (相等)
- `imm[31:0] = 4294967284`: 立即数 (-12 的补码表示)

- 测试结果：** ✓ 通过 成功检测到 x3==x5，PC 跳转到 0x18

The screenshot displays the Logic Analyzer interface. On the left, a list of signals is shown, including `clk`, `rst_n`, `pc[31:0]`, `pc_next[31:0]`, `instr[31:0]`, `rs1[4:0]`, `rs2[4:0]`, `imm[31:0]`, `rd[4:0]`, `rs1_data[31:0]`, `rs2_data[31:0]`, `alu_b[31:0]`, `alu_result[31:0]`, `alu_op[3:0]`, `alu_src`, `branch`, `branch_taken`, `mem_read`, `mem_to_reg`, `mem_write`, `imm_type[2:0]`, `alu_zero`, `addr[7:0]`, `registers[0:31][31:0]`, `memory[0:255][7:0]`, and `memory[0:255][7:0]`. The `alu_src` signal is currently selected. On the right, the timing diagram shows the digital values of these signals over time. The signals are represented by horizontal lines, with the selected `alu_src` signal highlighted in blue. The timing diagram shows that `alu_src` is a constant high signal (1). Other signals like `pc`, `pc_next`, `instr`, `rs1`, `rs2`, `imm`, `rd`, `rs1_data`, `rs2_data`, `alu_b`, `alu_result`, `alu_op`, `branch`, `branch_taken`, `mem_read`, `mem_to_reg`, `mem_write`, `imm_type`, `alu_zero`, `addr`, `registers`, `memory`, and `memory` show various digital patterns over time.

当前指令: `sw x6, 0(x2)` - 再次执行 `sw` 指令

- `pc[31:0] = 24 (0x18)`: 跳转后 PC 值正确
- `pc_next[31:0] = 28 (0x1C)`: 准备执行下一条指令
- `instr[31:0] = 6365219`: `sw` 指令机器码

- `branch = 0`: 当前指令不是分支指令
- `branch_taken = 0`: 无分支发生
- `alu_src = 1`: `sw` 指令使用立即数计算地址
- `mem_write = 1`: 执行内存写操作

测试结果： ✓ 通过 `beq` 指令正确实现条件跳转功能

测试总结

指令覆盖情况

指令类型	指令	测试地址	测试结果
I 型	<code>addi</code>	<code>PC=0x00</code>	✓ 通过
I 型	<code>lw</code>	<code>PC=0x04</code>	✓ 通过
R 型	<code>add</code>	<code>PC=0x08</code>	✓ 通过
R 型	<code>sub</code>	<code>PC=0x0C</code>	✓ 通过
R 型	<code>or</code>	<code>PC=0x10</code>	✓ 通过
I 型	<code>ori</code>	<code>PC=0x14</code>	✓ 通过
S 型	<code>sw</code>	<code>PC=0x18</code>	✓ 通过
R 型	<code>slt</code>	<code>PC=0x1C</code>	✓ 通过
I 型	<code>slti</code>	<code>PC=0x20</code>	✓ 通过
B 型	<code>beq</code>	<code>PC=0x24</code>	✓ 通过

控制信号验证

所有指令的控制信号均符合设计要求：

- ✓ `alu_op` - 根据指令类型正确选择 `ADD/SUB/OR/SLT` 操作
- ✓ `alu_src` - R 型=0(寄存器), I/S 型=1(立即数)
- ✓ `mem_read` - 仅 `lw` 指令=1
- ✓ `mem_write` - 仅 `sw` 指令=1
- ✓ `mem_to_reg` - 仅 `lw` 指令=1
- ✓ `reg_write` - R 型和 I 型(非 `sw`)=1, S 型和 B 型=0

- ✓ **branch** - 仅 beq 指令=1
- ✓ **branch_taken** - 仅 beq 条件满足时=1
- ✓ **imm_type** - I 型=0, S 型=1, B 型=2

数据通路验证

- ✓ **寄存器堆** - X0 恒为 0，其他寄存器读写正确
- ✓ **ALU 运算** - 加法、减法、按位或、有符号比较全部正确
- ✓ **立即数生成** - I/S/B 型立即数提取与符号扩展正确
- ✓ **内存访问** - lw 读取和 sw 写入均符合 Little Endian 格式
- ✓ **PC 更新** - 顺序执行+4，分支跳转 PC+offset 正确

存储器件数据展示验证

已在项目报告中提供具体的数据验证情况，正确实现。

性能分析

从波形图可以直观观察到单周期 CPU 的性能特点：

性能指标	数值	说明
CPI (每指令周期数)	1.0	单周期设计，每条指令恒定 1 个时钟周期
时钟周期	10ns	仿真设置，对应 100MHz 时钟
指令吞吐率	1 条/周期	每周期完成一条指令的完整执行
测试程序执行	10 个周期	10 条指令顺序执行（不含 beq 循环）

波形图体现的性能特点：

- 确定性执行**：每条指令在一个时钟周期内完成取指、译码、执行、访存、写回全部阶段
- 无流水线冒险**：单周期设计不存在数据冒险和控制冒险问题
- 分支延迟为零**：beq 指令在同一周期内完成比较和 PC 更新，下一周期直接执行目标指令
- 存储器访问效率**：lw/sw 在单周期内完成地址计算和数据传输

结论

所有 10 条指令（9 条必选 + 1 条附加分 beq）测试全部通过！

波形图验证要点总结：

- ✓ 采用功能仿真 (Behavioral Simulation) 验证 RTL 逻辑正确性
- ✓ **所有存储器件数据完整显示**：PM、DM、寄存器堆均可在波形中观察
- ✓ **波形具有代表性**：覆盖 R/I/S/B 四种指令类型的典型指令
- ✓ **体现程序性能**：CPI=1 的单周期特性、分支跳转的即时响应、存储器访问的正确时序